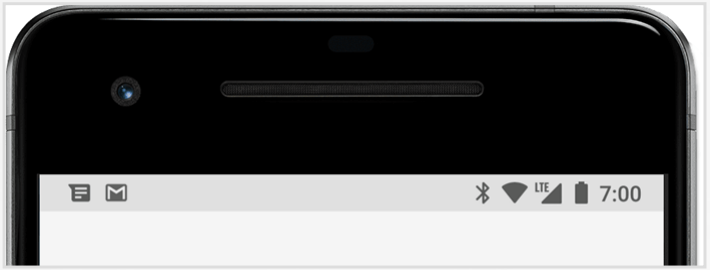
Software Development for Mobile Devices

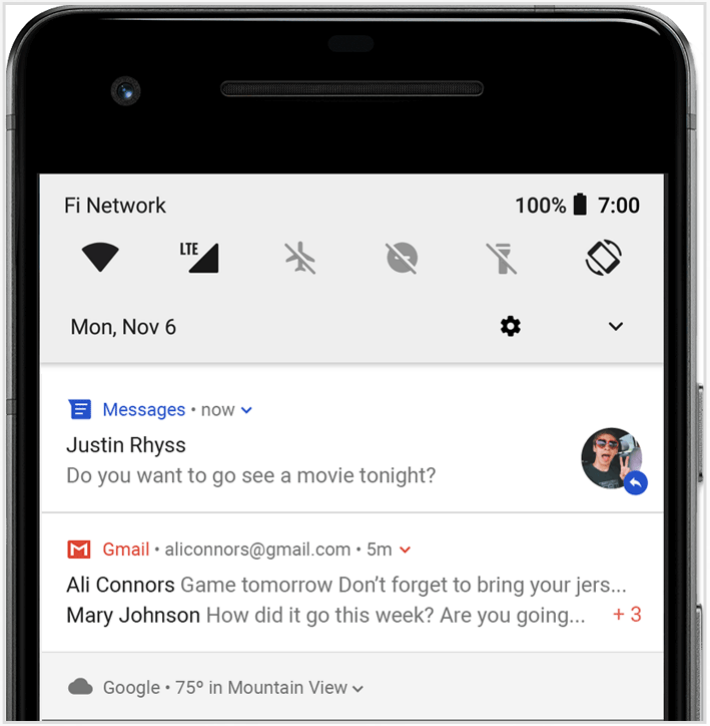
Notifications

Nguyen Le Hoang Dung
nlhdung@fit.hcmus.edu.vn

Notifications

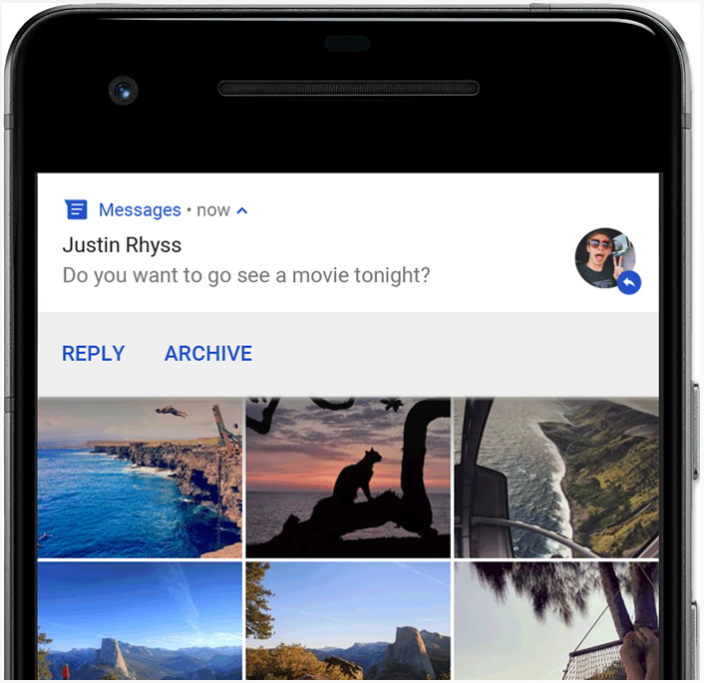
- What is a Notification?
  - A notification is a short message briefly displayed on the status line. It typically announces the happening of an special event for which a trigger has been set.
  - After opening the Notification Panel the user may choose to click on a selection and execute an associated activity





Notification Manager

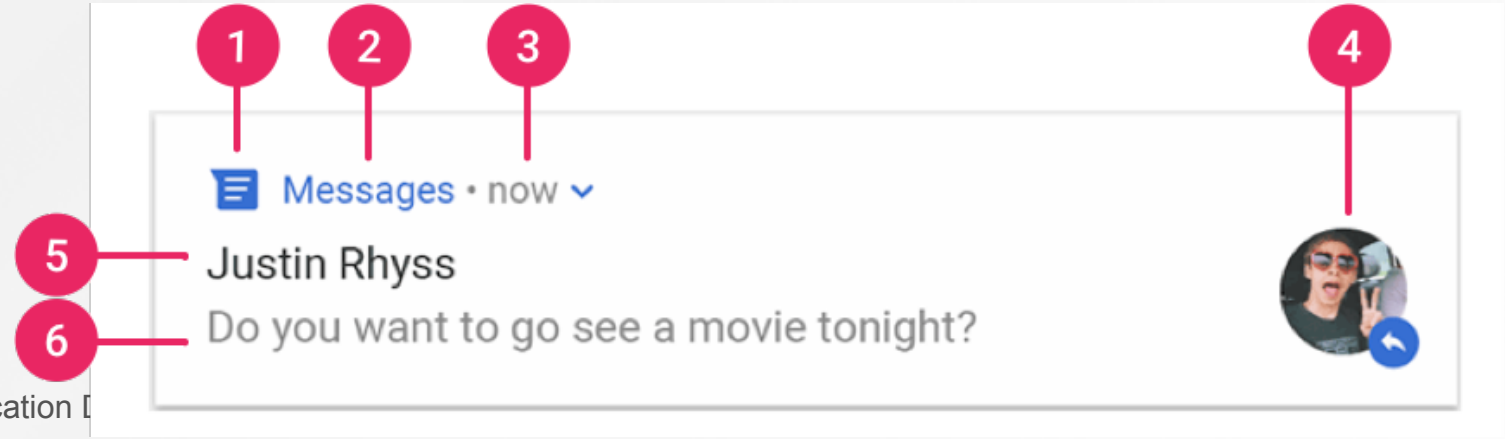
- This class notifies the user of events that happen in the background.
- Notifications can take different forms:
  - A persistent icon that goes in the status bar and is accessible through the launcher, (when the user selects it, a designated Intent can be launched)
  - Turning on or flashing LEDs on the device
  - Alerting the user by flashing the backlight, playing a sound, or vibrating



Heads-up notification

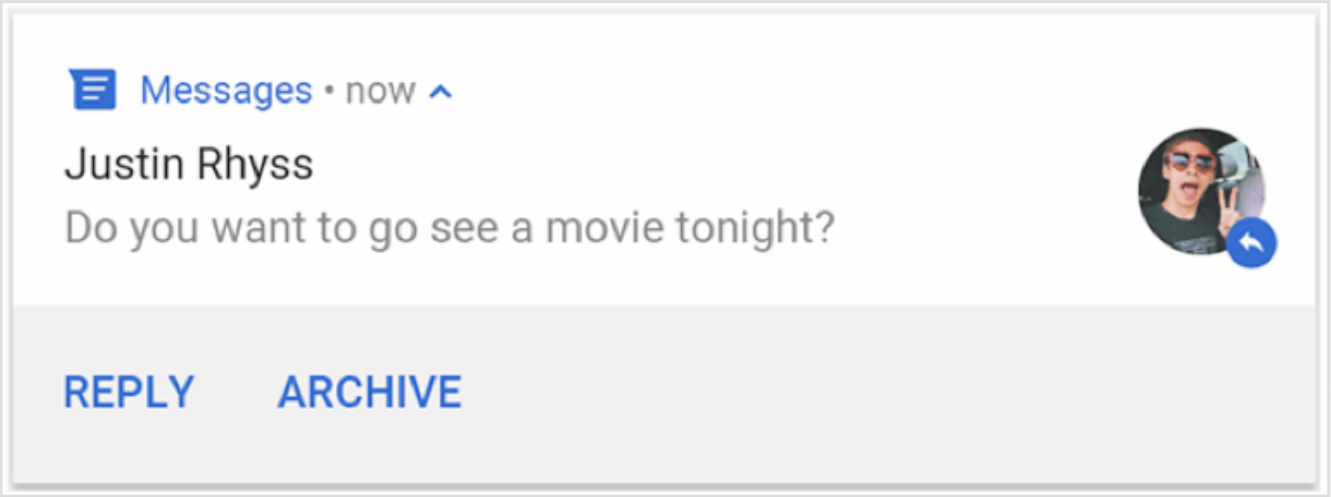
Notification anatomy

- The design of a notification is determined by system templates—your app simply defines the contents for each portion of the template. Some details of the notification appear only in the expanded view.
- The most common parts:
    1. Small icon: setSmallIcon().
    2. App name: This is provided by the system.
    3. Time stamp: This is provided by the system, but you can override with setWhen() or hide it with setShowWhen(false).
    4. Large icon: This is optional and set with setLargeIcon().
    5. Title: This is optional and set with setContentTitle().
    6. Text: This is optional and set with setContentText().

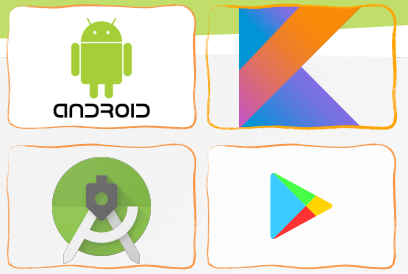


Notification actions

- Every notification should open an appropriate app activity when tapped.
- In addition to this default notification action, you can add action buttons that complete an app-related task from the notification (often without opening an activity)
  - Starting in Android 7.0 (API level 24), you can also add an action to reply to messages or enter other text directly from the notification.
  - Starting in Android 10 (API level 29), the platform can automatically generate action buttons with suggested intent-based actions.



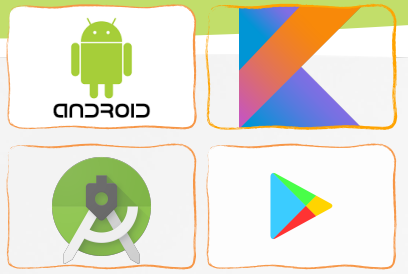
Notification actions



- On Android 12 (API level 31) and higher, you can configure a notification action such that the device must be unlocked for your app to invoke that action, no matter what workflow the action launches using setAuthenticationRequired()

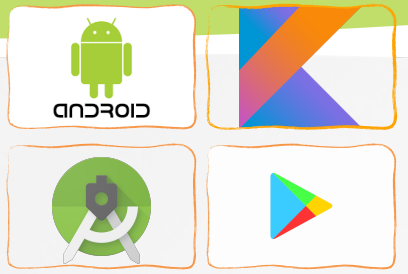
```
val moreSecureNotification = Notification.Action.Builder(...)

    //This notification always requests authentication
    // when invoked from a lock screen.
    .setAuthenticationRequired(true)
    .build()
```

Notification channels



- From Android 8.0 (API 26), all notifications must be assigned to a channel or it will not appear.
- By categorizing into channels, users can disable specific notification channels (instead of disabling *all* your notifications), and users can control the visual and auditory options for each channel
  - Users can also long-press a notification to change behaviors for the associated channel.
- An app can also create notification channels in response to choices made by users of your app. For example, you may set up separate notification channels for each conversation group created by a user in a messaging app.

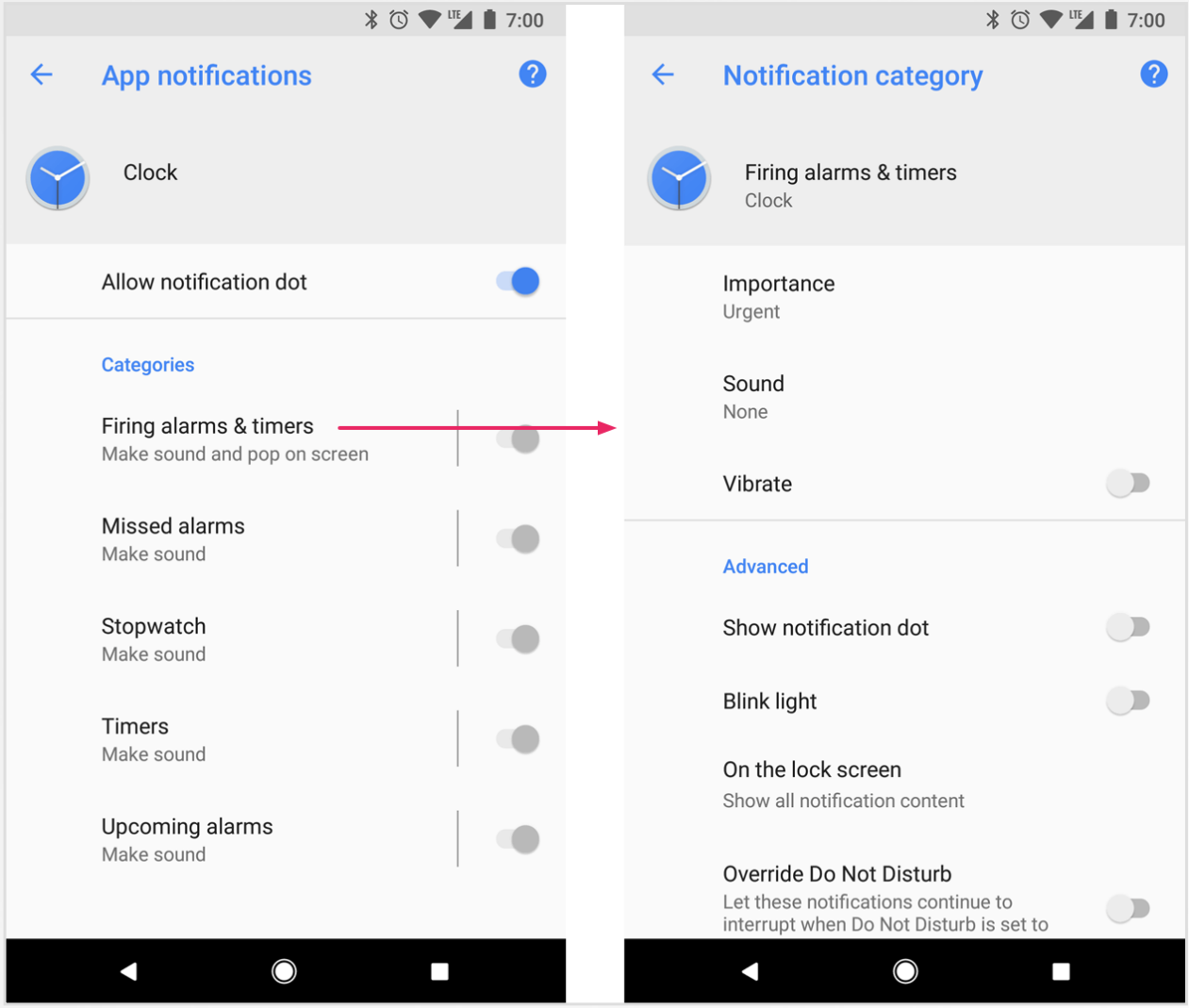
Notification channels



- The channel is also where you specify the importance level for your notifications (Android 8.0 and higher).
  - **Urgent**: makes a sound and appears as a heads-up notification.
  - **High**: makes a sound.
  - **Medium**: makes no sound.
  - **Low**: makes no sound and doesn't appear in the status bar.

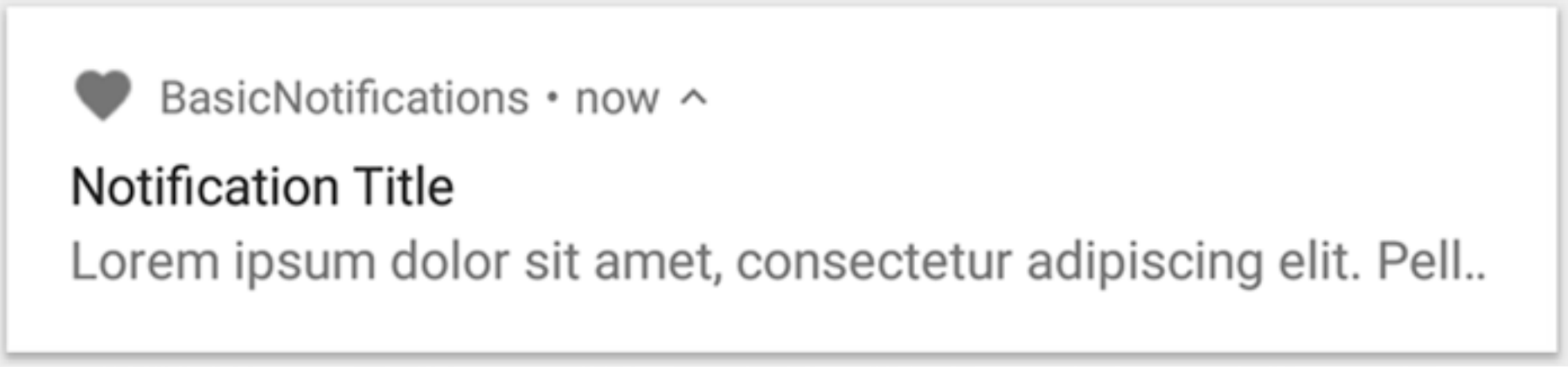
  => all notifications posted to the same notification channel have the same behavior.

Notification channels



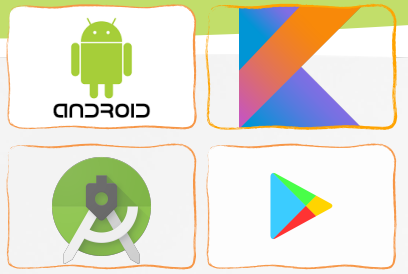
Create a Notification

- A notification in its most basic and compact form (collapsed form) displays an icon, a title, and a small amount of content text



- NotificationCompat.Builder class allows easier control over all the flags, as well as help constructing the typical notification layouts:
  - setSmallIcon(); setContentTitle(); setContentText()
  - priority, set by setPriority() (Android 7.1 and lower)
  - channel (Android 8.0 and higher)

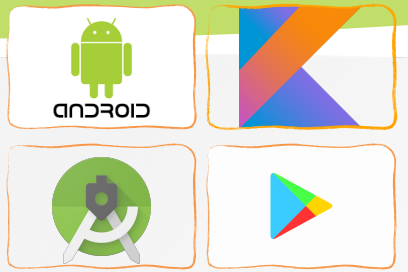
Create a Notification



- NotificationManager: notify the user of events that happen.
- NotificationChannel: a representation of settings that apply to a collection of similarly themed notifications

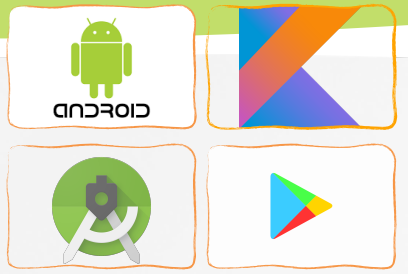
```
val CHANNEL_ID = "sampleChannelID"

val myBitmap =
    BitmapFactory.decodeResource(resources, R.drawable.notification_icon)
var builder = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.notification_icon)
    .setLargeIcon(myBitmap)
    .setContentTitle("My notification")
    .setContentText("Much longer text that cannot fit one line...")
    //By default, the notification's text content is truncated to fit one line
    .setStyle(NotificationCompat.BigTextStyle()
        .bigText("Much longer text that cannot fit one line..." +
                "\nMuch longer text that cannot fit one line..."))
    //.setStyle(NotificationCompat.BigPictureStyle().bigPicture(myBitmap))

    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
```

Create a Notification



```
private fun createNotificationChannel() {
    // Create the NotificationChannel, but only on API 26+ because
    // the NotificationChannel class is new and not in the support library
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        val name = getString(R.string.channel_name)
        val descriptionText = getString(R.string.channel_description)
        val importance = NotificationManager.IMPORTANCE_DEFAULT
        val channel = NotificationChannel(CHANNEL_ID, name, importance)
            .apply {
            description = descriptionText
        }

        // Register the channel with the system
        val notificationManager: NotificationManager =
            getSystemService(Context.NOTIFICATION_SERVICE) as
NotificationManager

        notificationManager.createNotificationChannel(channel)
    }
}
```

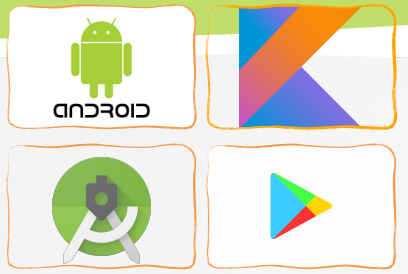
Create a Notification



- Set the notification's tap action

```
// Create an explicit intent for an Activity in your app
val intent = Intent(this, LocalNotificationActivity::class.java)
    .apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or
            Intent.FLAG_ACTIVITY_CLEAR_TASK
}
val pendingIntent: PendingIntent = PendingIntent.getActivity(this, 0,
    intent, PendingIntent.FLAG_IMMUTABLE)

var builder = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle("My notification")
    .setContentText("Much longer text that cannot fit one line...")
    .setStyle(NotificationCompat.BigTextStyle()
        .bigText("Much longer text that cannot fit one line..." +
          "\nMuch longer text that cannot fit one line..."))
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    // Set the intent that will fire when the user taps the notification
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)
```

Create a Notification



- ***setAutoCancel()***: automatically removes the notification when the user taps it.
- ***setFlags()***: preserve the user's expected navigation experience after they open your app via the notification:
  - An activity that exists exclusively for responses to the notification. The activity starts a new task instead of being added to your app's existing task and back stack. This is the type of intent created in the sample above.
- Common Flags for ***`PendingIntent`***:
  - FLAG_ACTIVITY_NEW_TASK:
    - Launches a new task (or back stack) if no matching task exists.
    - If a matching task already exists, it brings that task to the foreground and starts the activity on top.
  - FLAG_ACTIVITY_CLEAR_TOP:
    - If the activity already exists in the current task, all activities above it in the stack are cleared, and the existing activity is reused.
    - Useful for avoiding multiple instances of the same activity.
  - FLAG_ACTIVITY_SINGLE_TOP:
    - If the activity is already at the top of the stack, it will not be recreated. Instead, the ***`onNewIntent`*** method is triggered.
  - FLAG_UPDATE_CURRENT:
    - Updates an existing ***`PendingIntent`*** with new extras and keeps it valid

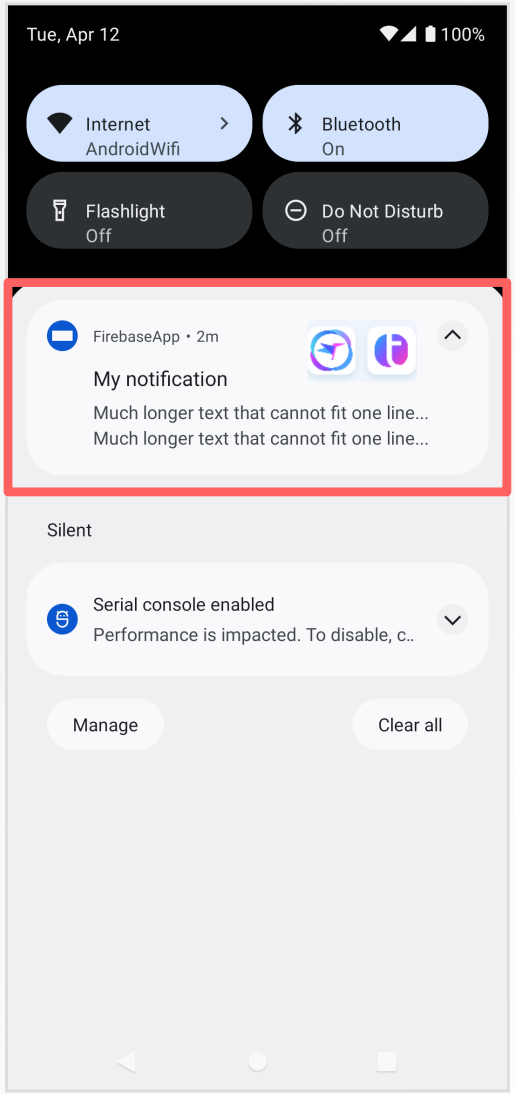
Create a Notification

PendingIntent Flag	Behavior
FLAG_UPDATE_CURRENT	Updates the Intent data if the PendingIntent already exists.
FLAG_CANCEL_CURRENT	Cancels the current PendingIntent and creates a new one.
FLAG_NO_CREATE	Does not create a new PendingIntent. Returns null if it does not already exist (check if PendingIntent is existed)
FLAG_IMMUTABLE	Ensures the Intent cannot be modified.
FLAG_ONE_SHOT	The PendingIntent can only be used once.

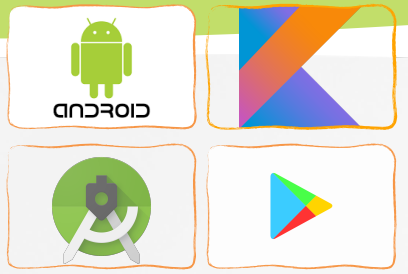
Create a Notification

- Show the notification with notification ID
  - We'll need notification ID later if we want to update or remove the notification.

```
val notificationId = 0
with(NotificationManagerCompat.from(this)) {
    //NotificationId is a unique int
    // for each notification
    notify(notificationId, builder.build())
}
```

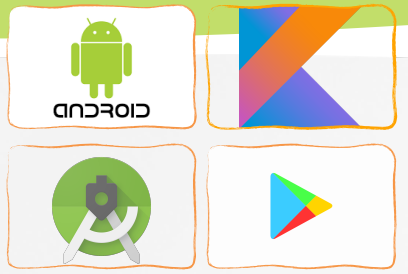


Create a Notification
Start an Activity from a Notification



- ***Regular activity***
  - This is an activity that exists as a part of your app's normal UX flow. So when the user arrives in the activity from the notification, the new task should include a complete back stack, allowing them to press *Back* and navigate up the app hierarchy.
- ***Special activity***
  - The user only sees this activity if it's started from a notification. In a sense, this activity extends the notification UI by providing information that would be hard to display in the notification itself.
  - This activity does not need a back stack.

Create a Notification

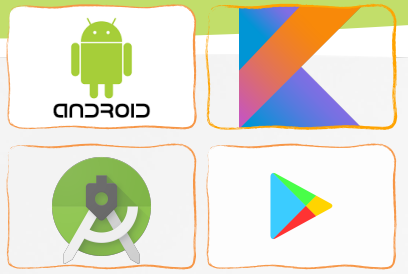


- ***Special activity***

```
val intent = Intent(this, LocalNotificationActivity::class.java)
    .apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or
            Intent.FLAG_ACTIVITY_CLEAR_TASK
}
intent.putExtra("key1", "value1")
val pendingIntent: PendingIntent = PendingIntent.getActivity(this, 0,
    intent, PendingIntent.FLAG_MUTABLE)

//…
```

```
class LocalNotificationActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_local_notification)

        Log.d("LocalNotificationActivity", intent.getStringExtra("key1").toString())
    }
}
```

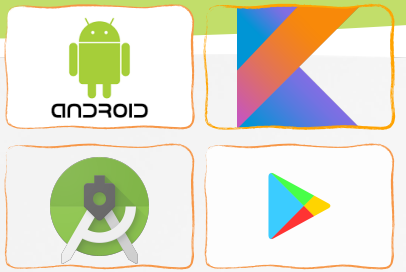
Create a Notification



- ***Regular activity***
  - Add the expected activity
  - Define the natural hierarchy for your activities by adding the android:parentActivityName attribute to each <activity> element in your app manifest file
  - Build a PendingIntent with a back stack

```
<activity
    android:name=".MainActivity"
    android:exported="true">
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<activity
    android:name=".LocalNotificationSubActivity"
    android:parentActivityName=".MainActivity"/>
```

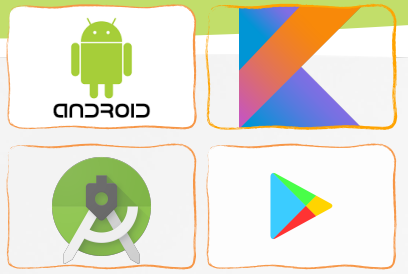
Create a Notification



- ***Regular activity***

```
// Create an Intent for the activity you want to start
val resultIntent = Intent(this, LocalNotificationSubActivity::class.java)
// Create the TaskStackBuilder
val stackBuilder = TaskStackBuilder.create(this)
stackBuilder.addNextIntentWithParentStack(resultIntent)
val resultPendingIntent: PendingIntent? = stackBuilder.getPendingIntent(0,
    PendingIntent.FLAG_UPDATE_CURRENT or PendingIntent.FLAG_IMMUTABLE)

var builder = NotificationCompat.Builder(this, CHANNEL_ID)
    .setSmallIcon(R.drawable.notification_icon)
    .setContentTitle("My notification")
    .setContentText("Much longer text that cannot fit one line...")
    .setStyle(NotificationCompat.BigTextStyle()
        .bigText("Much longer text that cannot fit one line..." +
                "\nMuch longer text that cannot fit one line..."))
    .setPriority(NotificationCompat.PRIORITY_DEFAULT)
    // Set the intent that will fire when the user taps the notification
    .setContentIntent(resultPendingIntent)
    .setAutoCancel(true)

with(NotificationManagerCompat.from(this)) {
    notify(0, builder.build())
}
```

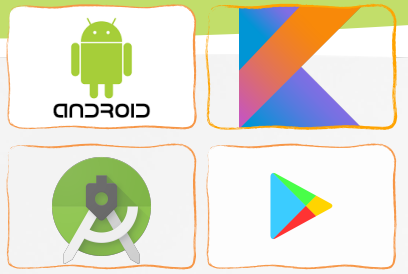
Create a Notification
Lock screen visibility



(With ***builder*** object)

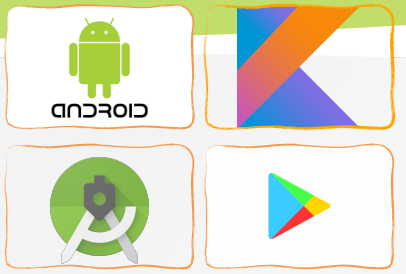
- setVisibility() : control the level of detail visible in the notification from the lock screen
  - VISIBILITY_PUBLIC shows the notification's full content.
  - VISIBILITY_SECRET doesn't show any part of this notification on the lock screen.
  - VISIBILITY_PRIVATE shows basic information, such as the notification's icon and the content title, but hides the notification's full content.
- With VISIBILITY_PRIVATE, we can also provide an alternate version of the notification content which hides certain details.
  - For example, an SMS app might display a notification that shows "*You have 3 new text messages*", but hides the message contents and senders.
  - Attach the alternative notification to the normal notification with setPublicVersion().

Create a Notification
Lock screen visibility



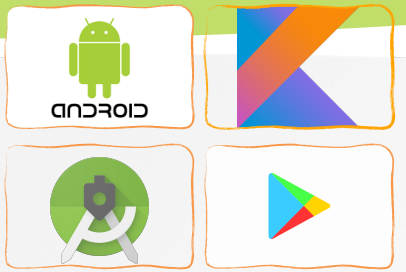
- ***Update a notification*** with NotificationManagerCompat.notify()
  - Passing it a notification with the same ID you used previously.
  - If the previous notification has been dismissed, a new notification is created instead.
  - You can optionally call setOnlyAlertOnce() so your notification interupts the user (with sound, vibration, or visual clues) only the first time the notification appears and not for later updates.
  - **Caution:** Android applies a rate limit when updating a notification. If you post updates to a notification too frequently (many in less than 1 second), the system might drop some updates.

Create a Notification
Lock screen visibility



- ***Remove a notification***: notifications remain visible until one of the following happens:
  - The user dismisses the notification.
  - The user clicks the notification, and you called setAutoCancel()
  - You call cancel() for a specific notification ID
  - You call cancelAll()
  - If you set a timeout when creating a notification using setTimeoutAfter()

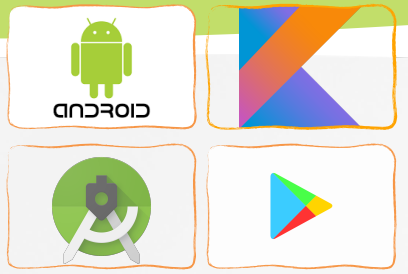
Practice



- Try to send messages with the Local Notifications

Q&A

Reference



- https://developer.android.com/guide/topics/ui/notifiers/notifications
- Peter Späth, Learn Kotlin for Android Development, 2019
- Ted Hagos, Learn Android Studio 3 with Kotlin, 2018
- Peter Späth, Pro Android with Kotlin, 2019
- Milos Vasic, Mastering Android Development with Kotlin, 2017